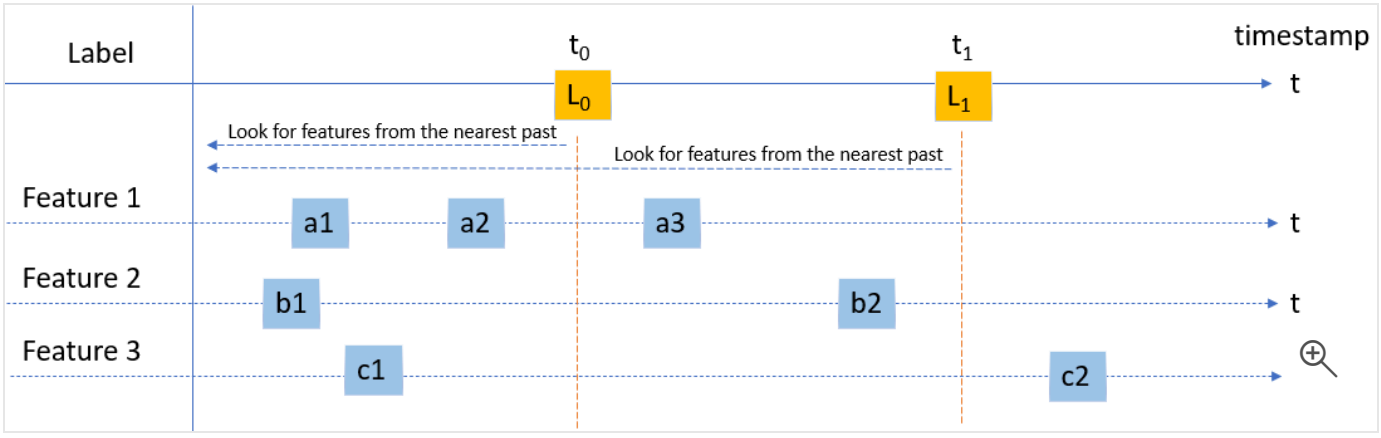
Offline feature retrieval using a point-in-time join

Understanding the point-in-time join

A *point-in-time* or temporal join helps prevent data leakage. In the model training process, data leakage , or target leakage, involves using information that isn't expected to be available at prediction time. Target leakage causes the predictive scores and metrics to overestimate the utility of the model when the model runs in a production environment. This article explains data leakage.

The next illustration explains how feature store point-in-time joins work:

- The observation data has two labeled events: `L0` and `L1`. The two events occurred at times `t0` and `t1` respectively.
- A training sample is created from this observation data, with a point-in-time join. For each observation event, the feature value from its most recent previous event time (`t0` and `t1`) is joined with the event.



This screenshot shows the output of a function named `get_offline_features`. That function executes a point-in-time join:

Feature 1	Feature 2	Feature 3	Label
a2	b1	c1	L_0
a3	b2	c1	L_1

Parameters used by point-in-time join

In the feature set specification, these parameters affect the result of the point-in-time join:

- `source_delay`
- `temporal_join_lookback`

Both parameters represent a duration, or time delta. For an observation event that has a timestamp `t` value, the feature value with the latest timestamp in the window `[t - temporal_join_lookback, t - source_delay]` joins to the observation event data.

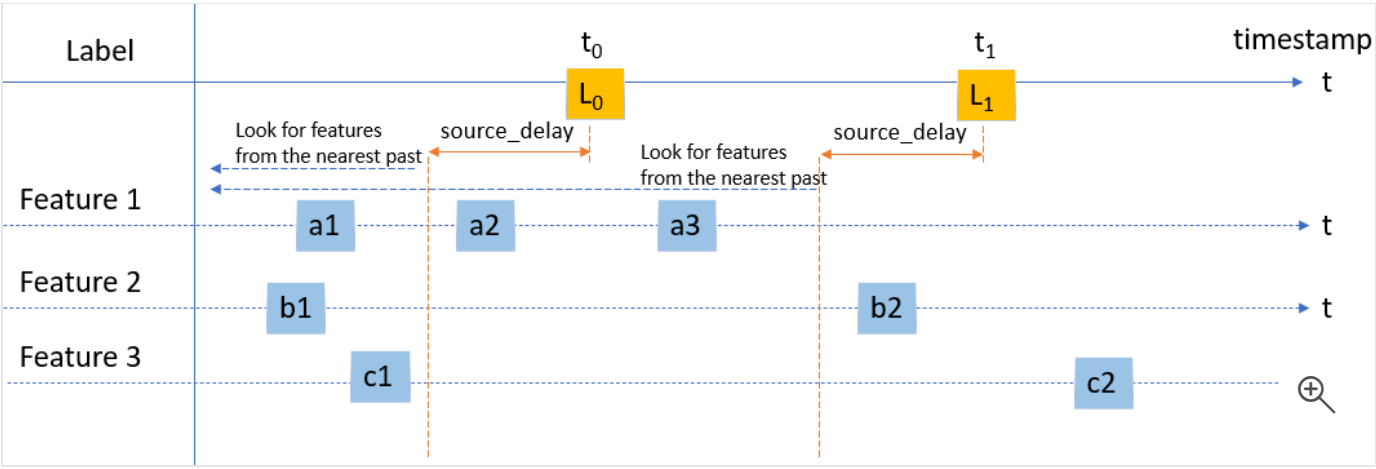
The `source_delay` property

The `source_delay` property indicates the acquisition time delay at the moment that data is ready for consumption. The time value at that moment is compared to the time value at the moment of generation of that data. An event that happened at time `t` lands in the source data table at time `t + x`, due to the latency in the upstream data pipeline. The `x` value is the source delay.

Source delay can lead to Data leakage :

- When you train a model with offline data, without consideration of source delay, the model uses feature values from the nearest past.
- When you deploy a model to a production environment, the model only uses feature values delayed by at least the amount of source delay time. As a result, the predictive scores degrade. To address source delay data leakage, the `source_delay` value in the point-in-time join is considered. To define the `source_delay` in the feature set specification, estimate the source delay duration.

In the same example, given a `source_delay` value, events `L0` and `L1` join with earlier feature values, instead of feature values in the nearest, most recent past.



This screenshot shows the output of the `get_offline_features` function that performs the point-in-time join:

Feature 1	Feature 2	Feature 3	Label
a1	b1	c1	L_0
a3	b1	c1	L_1

If you don't set the `source_delay` value in the feature set specification, its default value is `0`. This value means that no source delay is involved. The `source_delay` value is also considered in recurrent feature materialization. For more information about feature set materialization, see Feature set materialization concepts.

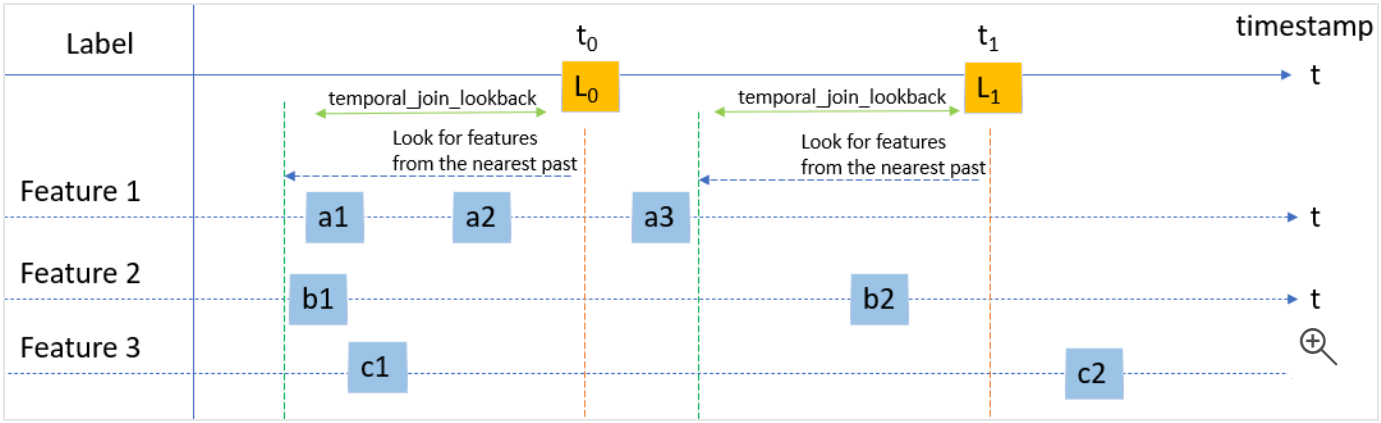
The `temporal_join_lookback`

A point-in-time join searches for previous feature values that are closest in time to the observation event. The join might fetch a feature value that is too early if the feature value didn't update since that earlier time. This situation can lead to problems:

- A search for feature values with time values that are too early impacts the query performance of the point-in-time join.
- Feature values produced too early are stale. As model input, these values can degrade model prediction performance.

To prevent retrieval of feature values with time values that are too early, set the `temporal_join_lookback` parameter in the feature set specification. This parameter controls the earliest feature time values the point-in-time join accepts.

In the same example, given `temporal_join_lookback`, event `L1` only gets joined with feature values in the past, up to `t1 - temporal_join_lookback`.



This screenshot shows the output of the `get_offline_features` function. This function performs the point-in-time join:

Feature 1	Feature 2	Feature 3	Label
a2	b1	c1	L_0
none	b2	none	L_1

When you set `temporal_join_lookback`, set its duration time greater than `source_delay` to get nonempty join results. If you don't set the `temporal_join_lookback` value, its default value is infinity. It looks back as far as possible during the point-in-time join.

Next steps

- Tutorial 1: Develop and register a feature set with managed feature store
- GitHub Sample Repository

Last updated on 01/30/2026